

Prog. Exercises

```
def double_factorial(n: int) -> int:
    """Returns all (n-2) * (n-4) * ... * 2 if n is odd
    and (n-1) * (n-3) * ... * 1 if n is even"""
    if n > 0:
        return double_factorial(n-2) * n
    else:
        if n == 0:
            return 1
        else:
            return double_factorial(n-2) * n
```

```
def logarithm(n: int) -> int:
    """Returns the log base 2 of n"""
    if n > 0:
        return logarithm(n//2) + 1
    else:
        return 0
```

```
def gcd(m: int, n: int) -> int:
    """Returns the greatest common divisor of m and n using Euclid's algorithm"""
    if m > 0:
        return gcd(n, m % n)
    else:
        return m
```

```
def first_digit(n: int) -> int:
    """Returns the first digit of the decimal representation of n"""
    if n > 0:
        return first_digit(n//10)
    else:
        return n
```

```
def member(x: any, v: list) -> bool:
    """Returns whether v contains x"""
    if x in v:
        return True
    else:
        return False
```

```
def member(x: any, v: list) -> bool:
    """Returns whether v contains x"""
    if x in v:
        return True
    else:
        return False
```

```
def intersection(v: list, w: list) -> list:
    """Returns a list that contains all elements that occur in both v and w"""
```

from functools import reduce

```
def sum_of_squares(l: list) -> int:
    return reduce(lambda x, y: x+y, l)
```

```
def length(l: list) -> int:
```

```
def square(n: int) -> list[int]:
    return [n**2 for i in range(1, n+1)]
```

```
def square(n: int, v: list[int]) -> list[int]:
    return [n**2 for i in v]
```

```
def position(n: int, v: list[int]) -> list[int]:
    return [i for i, v in enumerate(v) if v == n]
```

```
def count(n: int, v: list[int]) -> int:
    return len([i for i in v if i == n])
```

```
def count(n: int, v: list[int]) -> list[int]:
    return [i for i in v if i == n]
```

```
def bisect(a: float, b: float, eps: float) -> float:
```

```
if b - a < eps:
    return a
c = (a+b)/2
if f(a) * f(c) < 0:
    return bisect(a, c, eps)
else:
    return bisect(c, b, eps)
```

```
def member(x: any, v: list, i: int) -> bool:
    """Returns whether v contains x at index i"""
    if i < len(v):
        return v[i] == x
    else:
        return False
```

from dataclasses import dataclass

@dataclass

class Point:

x: int

y: int

p = Point(3, 5)

x_coord = p.x

y_coord = p.y

from dataclasses import dataclass

@dataclass

class TimeStamp:

hours: int

minutes: int

seconds: int

```
def make_timestamp(hours: int = 0, minutes: int = 0, seconds: int = 0) -> TimeStamp:
```

"""Returns a timestamp with the given time input"""

```
return TimeStamp(hours, minutes, seconds)
```

```
def valid(hours: int, minutes: int, seconds: int) -> bool:
```

"""Returns whether the given time is a valid time"""

"""valid(24, 6, 70)"""

False

return (0 <= hours < 24 and

0 <= minutes < 60 and

0 <= seconds < 60)

```
t1 = make_timestamp(2, 4, 7)
```

t1.hours = 6

t1 = t3

skip_hour(t1)

```
def skip_hour(t: TimeStamp) -> None:
```

"""Skips an hour in t"""

if t.hours == 23:

t.hours = 0

else:

t.hours = t.hours + 1

```
def skip_second(t: TimeStamp) -> None:
```

"""Skips a second in t"""

if t.second == 59:

t.second = 0

skip_minute(t)

else:

t.second = t.second + 1

```
def skip_minute(t: TimeStamp) -> None:
```

"""Skips a minute in t"""

if t.minute == 59:

t.minute = 0

skip_hour(t)

else:

t.minute = t.minute + 1

0 1 2 3 5 8

```
def skip(t1: TimeStamp, t2: TimeStamp) -> None:
```

"""Adds t2 to t1"""

t1.h = t1.h + t2.h

t1.m = t1.m + t2.m

t1.s = t1.s + t2.s

if t1.s > 59:

t1.s = t1.s - 59

t1.m = t1.m + 1

if t1.m > 59:

t1.m = t1.m - 59

t1.h = t1.h + 1

if t1.h > 23:

t1.h = t1.h - 24

```
def equals(t1: TimeStamp, t2: TimeStamp) -> bool:
```

"""Checks whether t1 is equal to t2"""

return (t1.h == t2.h and

t1.m == t2.m and

t1.s == t2.s)

import math

from dataclasses import dataclass

@dataclass

class Point2D:

x: float

y: float

```
def make_point(x: float, y: float) -> Point2D:
```

"""Returns a new instance of Point2D"""

return Point2D(x, y)

```
def move(p: Point2D, dx: float, dy: float) -> None:
```

"""Moves p according to the vector (dx, dy)"""

p.x = p.x + dx

p.y = p.y + dy

```
def distance_to_origin(p: Point2D) -> float:
```

"""Returns the distance from p to origin (0,0)"""

return math.sqrt(p.x**2 + p.y**2)

from dataclasses import dataclass

@dataclass

class Polygon:

v: list[Point2D]

```
def make_polygon(v: list[Point2D]) -> Polygon:
```

"""Creates a polygon from the list of its vertices"""

return Polygon(v)

from dataclasses import dataclass

@dataclass

class SleepingArt:

products: list[Product]

```

shopping_cart = [Products]

def make_empty_cart() -> shopping_cart
    return []

def add(Product, shopping_cart) -> None
    """
    """
    shopping_cart.append(Product)

def remove_if_item_in_shopping_cart(item) -> bool:
    """
    """
    return item in shopping_cart

def remove_if_item_not_in_shopping_cart(item) -> bool:
    """
    """
    return item not in shopping_cart

```

```

def distance(p1: Point2D, p2: Point2D) -> float:
    """
    Returns the distance from p1 to p2
    """
    return sqrt((p2.x - p1.x)**2 + (p2.y - p1.y)**2)

def equals(p1: Point2D, p2: Point2D) -> bool:
    """
    Checks if p1 & p2 represent the same point
    """
    return sqrt((p2.x - p1.x)**2 + (p2.y - p1.y)**2) == 0

def equals(p1: Point2D, p2: Point2D) -> bool:
    """
    Checks if p1 & p2 represent the same point
    """
    return p1.x == p2.x and p1.y == p2.y

def copy(p: Point2D) -> Point2D:
    """
    Returns a copy of p.
    """
    return make_point(p.x, p.y)

def to_string(p: Point2D) -> str:
    """
    Returns a textual representation of p.
    """
    return f'p({p.x}, {p.y})'

```